

Capítulo 8: Datos de entrada del sistema

A. Interpretación académica del papel del Capítulo 8 dentro de la tesis

En una tesis de ingeniería aplicada como la presente, el **Capítulo 8 (Datos de entrada del sistema)** cumple la función de **definir, caracterizar y delimitar el conjunto de insumos** que el sistema requiere para operar de manera coherente con su objetivo técnico. En el caso de MuseIQ, dicho objetivo no es la localización centimétrica ni un asistente conversacional generalista; la propuesta se orienta a **mediación cultural situada** durante la visita presencial mediante reconocimiento funcional de sala/zona, apoyo de orientación con sensores del smartphone, interacción por voz (TTS/STT) y una capa de IA conversacional sustentada en recuperación (RAG).

Desde la lógica de diseño de sistemas (ingeniería de requisitos y arquitectura), este capítulo debe funcionar como un **“contrato de datos”** entre:

- el entorno físico del museo (infraestructura BLE basada en ESP32 y disposición espacial),
- el dispositivo del visitante (sensores, micrófono, altavoz, interfaz),
- y el conocimiento curatorial institucional (corpus y metadatos) que habilita respuestas contextualizadas.

Por ello, el alcance correcto del Capítulo 8 no se reduce a enumerar “tipos de datos” en abstracto. Debe establecer, con rigor ingenieril, al menos lo siguiente:

- **Qué datos entran** (señales, parámetros, metadatos, contenidos), en qué modalidad (sensorial, textual, estructural) y con qué granularidad (sala/zona/pieza).
- **De dónde provienen** (BLE/ESP32, sensores del smartphone, usuario vía voz o texto, repositorio curatorial del museo).
- **Cómo se describen y organizan lógicamente** (campos, identificadores, relaciones, niveles de agregación, metadatos mínimos).
- **Qué rol funcional cumplen** dentro de la propuesta (activación de contenido, desambiguación contextual, consulta RAG, trazabilidad).
- **Qué criterios mínimos de calidad y pertinencia** deben cumplir para sostener confiabilidad operativa (datos de contexto robustos; corpus curatorial gobernado y trazable), con remisión explícita a capítulos posteriores cuando corresponda.

En términos narrativos, este capítulo debe cerrar la transición entre lo ya desarrollado (fundamentos y tecnologías base hasta el Capítulo 7) y lo que sigue (diseño y despliegue del sistema), asegurando que el diseño posterior no sea interpretado como una “implementación improvisada”, sino como la consecuencia lógica de insumos bien definidos.

B. Justificación de por qué ese enfoque es el correcto según la plantilla y el sílabo

La orientación del Capítulo 8 hacia **estructura, variables, fuentes y parámetros de entrada** se justifica por dos razones académicamente directas:

Primero, la plantilla de tesis ubica el Capítulo 8 inmediatamente antes del capítulo de diseño y despliegue. Esa localización estructural implica que el capítulo actúa como **base de especificación**: define insumos antes de describir arquitectura y despliegue. En el campo de sistemas con componentes sensoriales y de IA, **no es metodológicamente válido** describir arquitectura (cap. de diseño) sin fijar antes el conjunto de datos que alimenta sus módulos.

Segundo, el sílabo del Taller de Investigación plantea como exigencia del informe identificar “**variables, parámetros y señales clave**” del sistema, y además explicitar procesos críticos como “**adquisición de datos**” dentro de la arquitectura general. Por tanto, el capítulo debe enfocarse en los datos como insumos (no en resultados ni en pruebas). Esta lectura también evita un solapamiento conceptual con:

- el capítulo de **Diseño y Despliegue** (donde corresponde explicar flujos, módulos, integración e implementación),
- y el capítulo de **Normas Técnicas / gestión de datos** (donde corresponde formalizar políticas y buenas prácticas de tratamiento, seguridad, retención, etc.).

En síntesis: el enfoque correcto del Capítulo 8 es una **especificación de entradas** (señales + contenidos + parámetros + metadatos), organizada por subsistema y por función, con criterios de calidad que habiliten el diseño posterior. Esta orientación es plenamente consistente con una tesis aplicada, cuya validez depende de que lo implementado sea trazable a insumos definidos.

C. Propuesta de estructura del capítulo con numeración tipo tesis

A continuación se propone una estructura con numeración coherente con capítulos previos y con el rol de “contrato de datos”:

- 8.1 Introducción y propósito del capítulo
- 8.2 Clasificación de los datos de entrada en MuseIQ
- 8.3 Datos de entrada para localización funcional por sala o zona (BLE/ESP32)
- 8.4 Datos de entrada para orientación y apoyo contextual (sensores del smartphone)
- 8.5 Datos de entrada para interacción visitante-sistema (voz y eventos de interfaz)
- 8.6 Datos curatoriales y metadatos para respuestas contextuales (RAG)
- 8.7 Organización lógica, identificadores y criterios mínimos de calidad de datos
- 8.8 Síntesis e implicancias para el diseño y despliegue

Esta estructura separa explícitamente: (i) datos de contexto físico (8.3–8.4), (ii) datos de interacción (8.5), (iii) datos curatoriales (8.6) y (iv) organización y calidad (8.7). Así se responde de forma directa a la función esperada del capítulo sin desplazarlo hacia implementación.

D. Desarrollo extenso del contenido en redacción académica, listo para adaptar a LaTeX

8.1 Introducción y propósito del capítulo

El desempeño de MuseIQ depende de la disponibilidad y calidad de un conjunto de datos de entrada heterogéneo, compuesto por señales de proximidad BLE, mediciones de sensores del smartphone, entradas del visitante por voz o texto y un corpus curatorial institucional. En una solución de mediación cultural situada, los datos no cumplen un rol accesorio: constituyen la base que permite inferir contexto

de visita y, en consecuencia, seleccionar contenidos pertinentes y producir respuestas consistentes con el entorno físico en el que se encuentra el visitante.

Este capítulo define los **datos e insumos mínimos** que MuseIQ requiere para operar, describiendo su procedencia, forma de representación, organización lógica y rol funcional en el sistema. Asimismo, se establecen criterios de calidad y pertinencia de los datos, entendidos como condiciones mínimas para garantizar confiabilidad operativa, trazabilidad y coherencia semántica durante el uso en visita presencial. La descripción se enfoca en entradas del sistema. Los algoritmos de fusión, clasificación, recuperación y generación se desarrollan en capítulos posteriores, pero su viabilidad queda condicionada por la especificación de datos presentada aquí.

8.2 Clasificación de los datos de entrada en MuseIQ

Para efectos de diseño, los datos de entrada de MuseIQ se clasifican en cuatro categorías principales, según su naturaleza y función:

(i) **Datos de contexto físico-espacial (proximidad y sala/zona).** Incluyen identificadores transmitidos por beacons BLE (implementados sobre ESP32) y mediciones de intensidad de señal recibida (RSSI) observadas por el smartphone. Estas entradas habilitan el reconocimiento funcional de sala, sin pretensión de precisión centimétrica. El estándar Bluetooth contempla el uso de RSSI como indicador de intensidad de señal recibida en el enlace, y las plataformas móviles exponen esta medición en los resultados de escaneo BLE. ¹

(ii) **Datos de orientación y movimiento (apoyo contextual).** Comprenden mediciones de sensores inerciales y magnéticos del smartphone para estimar orientación relativa (por ejemplo, ángulos de orientación o vectores de rotación), con el fin de reducir ambigüedad dentro de una misma sala y apoyar la selección contextual. La documentación de Android describe explícitamente el cómputo de orientación a partir de acelerómetro y sensor geomagnético mediante matrices de rotación y funciones de orientación. ²

(iii) **Datos de interacción (voz, texto y eventos de interfaz).** Incluyen audio de entrada (capturado por micrófono), transcripción STT, texto escrito, selección explícita de contenido en pantalla y preferencias del usuario (idioma, accesibilidad). En particular, la API de reconocimiento de voz en Android puede implicar transmisión de audio a servidores remotos, lo que convierte estos datos en altamente sensibles desde el punto de vista de tratamiento y minimización. ³

(iv) **Datos curatoriales y metadatos para RAG (conocimiento institucional).** Incluyen textos interpretativos, fichas de piezas, descripciones de sala, metadatos de colección y material educativo; además de metadatos estructurados (identificadores, idioma, autoría, fuentes, derechos, relación sala-pieza, etc.) que permiten indexación, recuperación y trazabilidad. La literatura de RAG plantea explícitamente un enfoque híbrido que combina un generador con una memoria no paramétrica consultable (índice recuperable), permitiendo incorporar evidencia recuperada en tareas intensivas en conocimiento. ⁴

Bajo esta clasificación, se distingue además una propiedad transversal: **dinámica temporal**. Parte de los datos se generan en tiempo real (RSSI, orientación, voz); otra parte es predominantemente estática o de actualización esporádica (mapa de salas, inventario de beacons, corpus curatorial). Esta distinción es relevante porque determina mecanismos de actualización, validación y control de versiones en el diseño posterior.

8.3 Datos de entrada para localización funcional por sala o zona (BLE/ESP32)

La localización funcional en MuseIQ depende de beacons BLE instalados en el museo (implementados con ESP32) y del escaneo BLE realizado por el smartphone del visitante. En este contexto, los datos de entrada principales son:

a) Identidad del beacon y datos transmitidos en advertising

Cada beacon debe transmitir un identificador detectable por el smartphone. A nivel de implementación, existen formatos ampliamente utilizados sobre BLE advertising, tales como iBeacon y Eddystone.

- En iBeacon, la identidad se define por un **UUID** más campos **major** y **minor**, usados para agrupar y diferenciar beacons dentro de un despliegue. La documentación de Apple describe esta identidad como compuesta por UUID/major/minor, y expone además RSSI y un valor de “proximity” estimado. ⁵
- En el ecosistema ESP32, se documenta una estructura típica de trama iBeacon con prefijo, UUID (16 bytes), major (2 bytes), minor (2 bytes) y un campo de potencia TX (1 byte), utilizado para aproximación de distancia por pérdida de trayectoria (path loss). ⁶
- En Eddystone, se define un conjunto de “frame types” (UID, URL, TLM, EID) para mensajes de proximidad y telemetría en BLE advertising. La especificación pública describe el protocolo como un formato de mensajes BLE para beacons de proximidad. ⁷

Independientemente del formato elegido, MuseIQ requiere como datos mínimos:

- `beacon_id` (identificador lógico interno),
- `beacon_protocol` (p. ej., iBeacon o Eddystone-UID),
- `beacon_identifier_payload` (campos concretos: UUID/major/minor o namespace/instance),
- `sala_id` o `zona_id` asignado,
- parámetros de transmisión relevantes (`tx_power` nominal y/o configuración de advertising si aplica).

b) Inventario de beacons y mapeo espacial

El sistema requiere un inventario (dataset de configuración) que relacione cada `beacon_id` con una sala o zona del museo. Este inventario constituye un dato de entrada esencial porque el RSSI por sí solo no tiene significado museográfico sin una correspondencia `beacon → espacio`.

A nivel de diseño lógico, el inventario debe contener como mínimo:

- Identificador único del beacon (`beacon_id`).
- Identificador físico o de broadcasting (según protocolo).
- Sala/zona asociada (`sala_id` , `zona_id`).
- Descripción breve de ubicación (texto de instalación para mantenimiento).
- Fecha de instalación y responsable (para trazabilidad operativa).
- Parámetros de emisión relevantes (p. ej., potencia/configuración).

Esta estructura habilita el reconocimiento de sala como un problema de inferencia contextual basado en un conjunto de señales observadas asociadas a espacios discretos.

c) Observaciones BLE registradas por el smartphone

En tiempo real, el smartphone obtiene resultados de escaneo BLE. En Android, el objeto de resultado de escaneo (ScanResult) expone explícitamente:

- RSSI recibido en dBm (`getRssi()`),
- timestamp de observación (`getTimestampNanos()`),
- y, cuando está disponible, potencia de transmisión (`getTxPower()`), junto con el scan record.

8

Por tanto, para MuseIQ se definen como entradas mínimas por observación:

- `timestamp` (idealmente normalizado al modelo temporal del sistema),
- `beacon_identifier` (extraído del advertising payload),
- `rssi_dbm`,
- `tx_power_dbm` (si está presente o si el protocolo/SDK lo expone),
- `scan_record` (cuando se requiere decodificación adicional).

En el caso del ESP32 como beacon, el esquema de advertising y scanning se enmarca en el rol GAP de broadcasting/observing y en las rutinas de device discovery. La documentación de Espressif describe el proceso de advertising y scanning y advierte límites técnicos relevantes, como el tamaño máximo de datos de advertising de 31 bytes en advertising packet y scan response (en configuraciones legacy), lo que condiciona la cantidad de información que puede incluirse en el payload. 9

d) Variables derivadas consideradas como “entradas funcionales”

En tesis de ingeniería aplicada, es habitual distinguir entre señal cruda y variable de entrada efectiva. En MuseIQ, por la naturaleza ruidosa del RSSI, el sistema típicamente transforma observaciones en variables de ventana temporal. Sin desarrollar algoritmos (reservados para capítulos posteriores), este capítulo fija que la entrada funcional para inferir sala/zona no es un único RSSI instantáneo, sino un conjunto agregable.

Se define así, a nivel conceptual:

- `rssi_b(t)` como RSSI del beacon `b` observado en el instante `t`.
- Un vector de observación en ventana `W`:

$$\mathbf{x}_W = [\phi(rssi_{b_1}), \phi(rssi_{b_2}), \dots, \phi(rssi_{b_n})]$$

donde $\phi(\cdot)$ representa estadísticos como media, mediana, máximo o percentiles por beacon en la ventana `W`.

La elección de ϕ y de `W` pertenece al capítulo de diseño, pero este capítulo establece la necesidad de diseñar entradas robustas ante fluctuaciones de señal, reflexiones y oclusiones propias de interiores.

8.4 Datos de entrada para orientación y apoyo contextual (sensores del smartphone)

En MuseIQ, los sensores del smartphone cumplen un rol de **apoyo contextual**: mientras BLE sugiere sala/zona probable, la orientación aporta información complementaria sobre hacia dónde se dirige o enfoca el visitante. Los datos de entrada se definen en tres niveles:

a) Señales sensoriales base

Android expone sensores de posición y orientación. La documentación técnica establece que la orientación puede inferirse combinando acelerómetro y sensor geomagnético, construyendo una

matriz de rotación (`getRotationMatrix`) y derivando ángulos de orientación (`getOrientation`).

10

En consecuencia, MuseIQ considera como señales base:

- `a(t) = [a_x, a_y, a_z]` aceleración (acelerómetro),
- `m(t) = [m_x, m_y, m_z]` campo magnético (magnetómetro),
- y, cuando se disponga (según dispositivo), sensores compuestos como rotation vector (que integra acelerómetro/magnetómetro/giroscopio) para una orientación más estable. 11

b) Variables de orientación derivadas

A partir de las señales base, se definen variables derivadas relevantes para MuseIQ:

- `azimuth(t)` : orientación respecto al norte (rotación en el plano horizontal),
- `pitch(t)` : inclinación,
- `roll(t)` : rotación lateral.

Estas variables no pretenden entregar orientación absoluta perfecta; su función es reducir ambigüedad contextual, por ejemplo, al priorizar contenido de una vitrina o pieza “en frente” del visitante dentro de una sala. La base metodológica para su cálculo se encuentra descrita por Android al explicar el uso combinado de sensores y matrices de rotación. 10

c) Parámetros y metadatos de adquisición

Para que la señal sea utilizable, el sistema requiere parámetros de adquisición como:

- frecuencia de muestreo efectiva o modo de reporte (según sensor),
- timestamps coherentes con el registro BLE,
- indicadores de disponibilidad (no todos los dispositivos tendrán el mismo set de sensores).

Estos parámetros son entradas de configuración dependientes del dispositivo, y se integran a la estructura de sesión para asegurar comparabilidad y depuración.

8.5 Datos de entrada para interacción visitante-sistema (voz y eventos de interfaz)

MuseIQ propone interacción multimodal, con énfasis en voz por accesibilidad. El sistema requiere describir entradas de usuario con una estructura formal que permita, posteriormente, enlazarlas con contexto espacial y con recuperación del corpus.

a) Audio y STT como entrada

La entrada primaria de voz es una señal de audio capturada por el micrófono del smartphone. En Android, el uso de SpeechRecognizer implica el permiso de grabación de audio y la propia documentación advierte que la implementación “probablemente” hace streaming de audio a servidores remotos para reconocimiento, razón por la cual no se recomienda para reconocimiento continuo por consumo de batería y ancho de banda. 3

Dado ello, el “dato de entrada” que MuseIQ consume funcionalmente no es el audio en bruto como almacenamiento permanente, sino:

- `utterance_id` (identificador de la solicitud de voz),
- `audio_capture_timestamp` (marca temporal),
- `stt_text` (transcripción),
- `stt_confidence` (si la plataforma lo provee),

- `language_code` estimado o definido para STT.

Por sensibilidad, el tratamiento de audio/transcripción debe guiarse por minimización de datos y consentimiento, y se formalizará en el capítulo de normas de gestión de datos, alineado con la legislación aplicable.

b) Texto y eventos de interfaz como entradas equivalentes

Para robustez, se considera equivalente funcional del STT:

- `typed_text` (texto escrito),
- `ui_event` (selección de sala, selección de pieza, activación manual de contenido).

En un sistema guiado por contexto, un evento de interfaz puede operar como señal de desambiguación (por ejemplo, si el usuario selecciona explícitamente una pieza cuando la orientación no es concluyente).

c) TTS como dependencia de datos de salida y configuración

Aunque TTS produce salida, su operación depende de entradas de configuración: idioma, voz, velocidad, volumen y segmentación del texto. La API TextToSpeech en Android define la síntesis como conversión de texto a voz, su inicialización y su control por colas (`QUEUE_ADD`, `QUEUE_FLUSH`), lo cual obliga a representar adecuadamente el texto a sintetizar y parámetros de voz como datos del sistema.

12

En consecuencia, MuseIQ requiere como datos de entrada de configuración:

- `tts_language_code`,
- `tts_voice_id` (si aplica),
- parámetros de locución (tasa, pitch),
- reglas de segmentación de respuesta (p. ej., longitud máxima por intervención).

8.6 Datos curatoriales y metadatos para respuestas contextuales (RAG)

La capa de IA conversacional en MuseIQ se justifica por la necesidad de obtener respuestas situadas y específicas, pero con control de factualidad y trazabilidad. En términos de datos de entrada, esto exige un corpus institucional y su representación adecuada para recuperación.

a) Corpus curatorial como insumo estructural

RAG se define como un enfoque que integra un modelo generativo (memoria paramétrica) con un repositorio consultable (memoria no paramétrica) mediante recuperación de evidencia relevante. El trabajo fundacional de Lewis et al. describe esta integración como un modelo que accede a un índice denso recuperable para tareas intensivas en conocimiento y reporta mejoras en especificidad y factualidad respecto a un modelo solo paramétrico. ⁴

En MuseIQ, el corpus curatorial debe ser entendido como un **dataset de entrada** con gobernanza institucional. Su composición mínima incluye:

- fichas de piezas (descripción, autoría/atribución, materialidad, época, procedencia),
- textos interpretativos (narrativas, contexto histórico-cultural),
- descripciones de sala (tema curatorial, recorrido sugerido, relaciones entre piezas),
- material educativo (glosarios, preguntas frecuentes, contenidos por nivel),
- información de visita relevante (accesibilidad, normas del espacio, etc.).

b) Metadatos culturales y estándares como referencia de modelado

La organización del corpus no debe depender exclusivamente de texto libre. En patrimonio cultural, existen marcos para estructurar información con fines de integración e intercambio:

- **CIDOC CRM** es una ontología de referencia para información de patrimonio cultural, reconocida como estándar ISO 21127 y orientada a la interoperabilidad semántica entre fuentes heterogéneas. ¹³
- **LIDO** se define como un formato (XML schema) para aportar información de objetos de colección para “resource discovery”, con soporte explícito para entornos multilingües mediante atributos de idioma en elementos. ¹⁴
- Como base mínima de catalogación interoperable, los **elementos Dublin Core** definen campos como título, creador, asunto, descripción, fecha e identificador, y recomiendan prácticas como uso de vocabularios controlados para “Subject” y codificación de fechas basada en un perfil de ISO 8601. ¹⁵

MuseIQ no requiere implementar exhaustivamente una ontología patrimonial compleja; sin embargo, sí necesita adoptar el principio de **metadatos mínimos consistentes**, suficientes para indexación, filtrado contextual y trazabilidad. En particular, para vincular contexto físico con conocimiento, cada pieza o unidad de contenido debe contener:

- `museo_id`,
- `sala_id` y/o `zona_id` asociada,
- `pieza_id` (identificador interno),
- `titulo`, `descripcion`,
- `keywords` (idealmente desde vocabularios controlados),
- `idioma`,
- `fuentes` y/o referencia curatorial institucional,
- `derechos` y condiciones de reutilización si se integran imágenes o recursos multimedia.

c) Vocabularios controlados y consistencia terminológica

La calidad semántica del corpus depende de consistencia en nombres de materiales, técnicas, periodos, estilos y agentes. En la práctica museística digital, vocabularios controlados como los Getty Vocabularies se presentan como recursos estructurados para artes visuales y patrimonio, con disponibilidad en formatos interoperables y orientación a catalogación e investigación (por ejemplo AAT, TGN, ULAN). ¹⁶

Sin imponer su adopción completa, se establece como criterio de entrada que MuseIQ debe privilegiar:

- listas controladas institucionales, o
- alineamiento parcial a vocabularios reconocidos, para evitar dispersión terminológica que degrade recuperación y aumente ambigüedad en respuesta.

d) Recursos multimedia e interop de imágenes

Si MuseIQ incorpora imágenes de piezas o recursos visuales (ya sea en la app o como apoyo a la respuesta), resulta pertinente considerar estándares de entrega. IIIF define APIs para describir y entregar imágenes y objetos digitales en web, incluyendo Image API y Presentation API, usados ampliamente en instituciones culturales para interoperabilidad. ¹⁷

En términos de datos de entrada, ello se traduce en almacenar:

- referencias a recursos (por ejemplo, `image_uri` o `manifest_uri`),
- metadatos de atribución,
- vínculo pieza–recurso.

8.7 Organización lógica, identificadores y criterios mínimos de calidad de datos

Dado que MuseIQ integra señales físicas y conocimiento curatorial, la organización de datos debe permitir unión consistente entre ambos mundos.

a) Principio de identificadores estables (IDs internos)

Se requiere definir identificadores internos estables para: museo, sala/zona, beacon, pieza (POI), documento curatorial y unidad recuperable (chunk). Este principio habilita:

- mapeo robusto beacon→sala,
- filtrado del corpus por sala o pieza,
- trazabilidad (qué contenido sustentó una respuesta),
- auditoría en evaluación posterior.

b) Modelo lógico mínimo de entidades y relaciones

Sin imponer un diseño de base de datos, se plantea un modelo lógico mínimo como estructura de datos de entrada:

- `Sala(sala_id, nombre, descripcion, piso, ...)`
- `Beacon(beacon_id, protocolo, payload_id, sala_id, tx_power, ubicacion_texto, ...)`
- `Pieza(pieza_id, sala_id, titulo, descripcion, etiquetas, ...)`
- `DocumentoCuratorial(doc_id, pieza_id/sala_id, idioma, tipo, texto, fuentes, derechos, version, ...)`
- `Chunk(chunk_id, doc_id, texto_segmentado, metadata_contextual, ...)`
- `EventoBLE(event_id, timestamp, beacon_id/payload_id, rssi_dbm, ...)`
- `EventoSensor(sensor_id, timestamp, azimuth, pitch, roll, ...)`
- `Consulta(query_id, timestamp, stt_text/typed_text, idioma, ...)`

Esta estructura explicita que las observaciones sensoriales no tienen valor museístico si no se relacionan con salas/piezas, y que el corpus textual no puede operar como base de RAG si no posee metadatos de contexto.

c) Criterios mínimos de calidad de datos (enfoque de requisitos)

En ingeniería de datos, la calidad no es un atributo abstracto; se convierte en requisitos verificables. ISO/IEC 25012 define un modelo general de calidad de datos para datos en formato estructurado dentro de sistemas computacionales, y lo orienta a definir requisitos, medidas y evaluaciones de calidad. ¹⁸

En MuseIQ, se establecen como criterios mínimos de calidad para datos de entrada:

- **Exactitud y consistencia (BLE/Inventario):** el inventario beacon–sala debe estar libre de duplicidades o asignaciones contradictorias. Un beacon no puede mapear a múltiples salas en el mismo intervalo operativo sin una regla explícita.
- **Actualidad (curatorial):** el corpus debe mantener control de versiones y fecha de actualización; la recuperación debe priorizar versiones vigentes cuando existan revisiones curatoriales.

- **Completitud (metadatos mínimos):** cada documento curatorial debe incluir al menos identificador, título, idioma, relación con sala/pieza y referencia de procedencia institucional. En ausencia de estos campos, la respuesta conversacional pierde trazabilidad.
- **Credibilidad y trazabilidad:** los contenidos deben estar respaldados por la institución o por fuentes curatoriales declaradas, ya que el propósito de RAG es reducir generación no fundamentada. ¹⁹
- **Seguridad y minimización (voz y sesión):** dado que el reconocimiento de voz puede implicar transmisión de audio, debe tratarse con criterios de minimización y consentimiento; se prefiere almacenar transcripción y metadatos mínimos operativos antes que audio crudo, salvo necesidad explícita para evaluación y con salvaguardas. ³

d) Pertinencia de datos y seguridad semántica en sistemas con RAG

En sistemas basados en recuperación y generación, existe un riesgo adicional: la entrada textual puede contener instrucciones maliciosas (“prompt injection”) que intenten desviar el comportamiento del modelo. OWASP identifica prompt injection como riesgo crítico en aplicaciones con LLMs, precisamente porque el sistema puede ser manipulado mediante entradas diseñadas para alterar instrucciones o provocar filtraciones. ²⁰

Para MuseIQ, esto refuerza un criterio de pertinencia:

- el corpus curatorial debe provenir de fuentes institucionales controladas,
- y las entradas de usuario (voz/texto) deben ser tratadas como datos no confiables que requieren diseño defensivo en el capítulo de arquitectura (por ejemplo, separación estricta entre instrucciones del sistema y contenido recuperado, y políticas de citación/justificación).

8.8 Síntesis e implicancias para el diseño y despliegue

En síntesis, MuseIQ requiere como entradas (i) un inventario de infraestructura BLE y su mapeo espacial, (ii) observaciones BLE y de sensores móviles con timestamps coherentes, (iii) entradas de interacción por voz/texto y preferencias, y (iv) un corpus curatorial estructurado y gobernado, acompañado de metadatos mínimos para recuperación contextual y trazabilidad.

La consecuencia directa para el diseño posterior es que la arquitectura debe:

- integrar adquisición de señales BLE y sensores con un modelo temporal consistente,
- definir mecanismos de inferencia de sala/zona basados en variables robustas (no solo lecturas instantáneas),
- y estructurar un pipeline de RAG donde el corpus sea tratable como memoria no paramétrica consultable, con control de calidad y mitigación de riesgos de seguridad semántica.

Estos aspectos se detallarán en el capítulo de diseño, pero su viabilidad depende de la especificación de entradas presentada en este capítulo.

E. Referencias sugeridas o utilizadas

Lewis, P. et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, NeurIPS 2020 / arXiv:2005.11401. ⁴

Karpukhin, V. et al., “Dense Passage Retrieval for Open-Domain Question Answering”, arXiv:2004.04906 (referencia técnica de recuperación densa utilizada en sistemas RAG). ²¹

Bluetooth SIG, Bluetooth Core Specification (documentación del estándar Bluetooth, incluyendo definiciones asociadas a RSSI y estructura general del enlace). 22

Documentación Android Developers, sensores de posición/orientación (uso de acelerómetro y sensor geomagnético, matriz de rotación y ángulos de orientación). 2

Documentación Android Developers, ScanResult BLE (RSSI en dBm, timestamp y campos asociados a resultados de escaneo). 8

Documentación Android Developers, SpeechRecognizer (reconocimiento de voz; advertencias sobre streaming de audio y consumo). 3

Documentación Android Developers, TextToSpeech (síntesis de voz y parámetros de operación). 12

Espressif, documentación BLE en ESP-IDF (visión general de API BLE, GAP/GATT, advertising/scanning). 23

Espressif, ejemplo/documentación de iBeacon en ESP32 (estructura de trama iBeacon y campos). 6

Documentación técnica de iBeacon (identidad por UUID/major/minor, y variables expuestas como RSSI). 5

Google, repositorio de especificación Eddystone (formatos de frame UID/URL/TLM/EID). 7

CIDOC CRM / ISO 21127 (ontología de referencia para intercambio de información de patrimonio cultural). 13

ICOM/CIDOC, LIDO (formato XML para describir objetos de colección y soporte multilingüe). 14

Dublin Core Metadata Initiative, Dublin Core Metadata Element Set v1.1 (campos mínimos, recomendación de vocabularios controlados e ISO 8601 para fechas). 15

ISO/IEC 25012:2008 (modelo de calidad de datos para datos estructurados en sistemas computacionales). 18

Getty Vocabularies (vocabularios controlados para catalogación de patrimonio y artes visuales). 16

IIIF (APIs para entrega e interoperabilidad de imágenes y objetos digitales en instituciones culturales). 17

Perú, Ley N.º 29733 de Protección de Datos Personales y su Reglamento (marco legal de tratamiento de datos personales aplicable al sistema). 24

OWASP, Top 10 for LLM Applications y material sobre prompt injection (riesgos de seguridad relevantes a sistemas con LLM/RAG y entradas no confiables). 20

F. Aspectos que convendría desarrollar después en capítulos posteriores

El Capítulo 8 fija el **qué** entra al sistema y con qué requisitos mínimos. Sin embargo, por coherencia con la estructura de tesis, los siguientes aspectos conviene desarrollarlos después:

En el capítulo de diseño y despliegue (arquitectura y operación):

- Diseño detallado del **pipeline de adquisición** (sincronización temporal BLE-sensores, manejo de latencias, ventanas de agregación, estrategias de escaneo y consumo energético en BYOD). ²⁵
- Selección y justificación del **método de inferencia de sala/zona** (reglas, clasificación, filtros), incluyendo parámetros (umbrales RSSI, smoothing) y criterios de robustez en interiores, coherentes con el objetivo de “reconocimiento funcional” y no precisión centimétrica. ¹
- Diseño del **modelo de indexación RAG**: segmentación del corpus (chunking), embeddings, almacenamiento en índice, filtros por metadatos (`sala_id` , `pieza_id`), y trazabilidad de evidencia recuperada hacia la respuesta. ²⁶
- Diseño de **estrategias de mitigación** frente a riesgos propios de LLM/RAG, especialmente prompt injection y sobreconfianza, incorporando validación de salidas y separación entre instrucciones y contenido recuperado. ²⁰

En el capítulo de normas técnicas / gestión de datos:

- Políticas formales de **minimización, consentimiento, retención y anonimización** aplicables a voz (audio/transcripción) y datos de sesión, alineadas con el marco legal peruano (Ley 29733 y Reglamento). ²⁷
- Estándares de seguridad y controles (por ejemplo, gestión de accesos a corpus, auditoría de cambios curatoriales, integridad del repositorio) y su relación con la operación institucional.

En el capítulo de validación o evaluación (si se incorpora como parte del proyecto):

- Protocolo de verificación de **calidad de datos** y procedimientos de actualización del inventario de beacons (mantenimiento, reasignaciones de sala, control de versiones del corpus), usando el modelo de calidad de datos como marco de evaluación. ¹⁸
- Diseño experimental para medir impacto de calidad de entradas sobre desempeño del sistema (p. ej., degradación por pérdida de beacons, interferencias, o variaciones de sensores).

¹ ²² Bluetooth Core Specification

https://www.bluetooth.com/wp-content/uploads/Files/Specification/HTML/Core-54/out/en/index-en.html?utm_source=chatgpt.com

² ¹⁰ Position sensors | Sensors and location | Android Developers

https://developer.android.com/develop/sensors-and-location/sensors/sensors_position?utm_source=chatgpt.com

³ SpeechRecognizer | API reference | Android Developers

https://developer.android.com/reference/android/speech/SpeechRecognizer?utm_source=chatgpt.com

⁴ ¹⁹ ²⁶ Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

https://papers.nips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html?utm_source=chatgpt.com

⁵ CLBeacon | Apple Developer Documentation

https://developer.apple.com/documentation/corelocation/clbeacon?language=objc&utm_source=chatgpt.com

- 6 [ESP32 Only] BLE AT Examples — ESP-AT User Guide documentation
https://docs.espressif.com/projects/esp-at/en/release-v2.2.0.0_esp8266/AT_Command_Examples/BLE_AT_Examples.html?utm_source=chatgpt.com
- 7 GitHub - google/eddystone: Specification for Eddystone, an open beacon format from Google
https://github.com/google/eddystone?utm_source=chatgpt.com
- 8 25 ScanResult | API reference | Android Developers
https://developer.android.com/reference/android/bluetooth/le/ScanResult?utm_source=chatgpt.com
- 9 Device Discovery - ESP32-H2 - — ESP-IDF Programming Guide latest documentation
https://docs.espressif.com/projects/esp-idf/en/latest/esp32h2/api-guides/ble/get-started/ble-device-discovery.html?utm_source=chatgpt.com
- 11 Sensor types | Android Open Source Project
https://source.android.com/docs/core/interaction/sensors/sensor-types?utm_source=chatgpt.com
- 12 TextToSpeech | API reference | Android Developers
https://developer.android.com/reference/android/speech/tts/TextToSpeech?utm_source=chatgpt.com
- 13 CIDOC CRM Short Intro | CIDOC CRM
https://cidoc-crm.org/cidoc-crm-short-intro?utm_source=chatgpt.com
- 14 What is LIDO? - ICOM DOCUMENTATION - ICOM DOCUMENTATION
https://cidoc.mini.icom.museum/working-groups/lido/what-is-lido/?utm_source=chatgpt.com
- 15 DCMI: Dublin Core™ Metadata Element Set, Version 1.1: Reference Description
https://www.dublincore.org/specifications/dublin-core/dces/2003-02-04/?utm_source=chatgpt.com
- 16 Getty Vocabularies (Getty Research Institute)
https://www.getty.edu/research/conducting_research/vocabularies/ulan/?utm_source=chatgpt.com
- 17 API Specifications - International Image Interoperability Framework™ — IIIF | International Image Interoperability Framework
https://iiif.io/api?utm_source=chatgpt.com
- 18 ISO/IEC 25012:2008 - Software engineering — Software product Quality Requirements and Evaluation (SQuaRE) — Data quality model
https://www.iso.org/standard/35736.html?utm_source=chatgpt.com
- 20 OWASP Top 10 for Large Language Model Applications | OWASP Foundation
https://owasp.org/www-project-top-10-for-large-language-model-applications/?utm_source=chatgpt.com
- 21 Dense Passage Retrieval for Open-Domain Question Answering | UCL NLP
https://nlp.cs.ucl.ac.uk/publications/2020-05-dense-passage-retrieval-for-open-domain-question-answering/?utm_source=chatgpt.com
- 23 Bluetooth® Low Energy - ESP32 - — ESP-IDF Programming Guide latest documentation
https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/bluetooth/bt_le.html?utm_source=chatgpt.com
- 24 27 Ley N.º 29733 - 2011 - Normas y documentos legales - Autoridad Nacional de Protección de Datos Personales - Plataforma del Estado Peruano
https://www.gob.pe/institucion/anpd/normas-legales/2018427-29733-2011?utm_source=chatgpt.com